

Memoria Técnica del Proyecto — Venus

Autor: David Berlanga Nieto

Repositorio: <https://github.com/davidberniet/peluqueria>

Proyecto: Venus — Sistema de Gestión de Peluquerías

Tipo: Trabajo de Fin de Grado (TFG)

Tecnología principal: Symfony 7.4 / PHP 8.2

Fecha: Mayo 2026

Índice

1. Descripción del Proyecto
 2. Arquitectura del Sistema
 3. Modelo de Datos
 4. Especificaciones Técnicas
 5. Manual de Despliegue
-

1. Descripción del Proyecto

1.1 Presentación

Venus es una aplicación web de gestión integral para peluquerías y centros de estética. El sistema permite a clientes reservar citas en línea de forma autónoma y a los administradores gestionar el negocio completo desde un panel de control dedicado, incluyendo el catálogo de servicios, los productos, los empleados y la configuración de horarios de cada local.

1.2 Objetivos

El proyecto persigue los siguientes objetivos funcionales y técnicos:

Objetivos funcionales:

- Ofrecer a los clientes un flujo de reserva de citas en línea guiado (selección de local → servicio → empleado → fecha y hora).
- Proporcionar un panel de administración completo para la gestión de citas, clientes, empleados, servicios, productos y locales.
- Automatizar el envío de recordatorios de cita por correo electrónico el día anterior a la cita.
- Permitir a los clientes valorar las citas completadas y gestionar su perfil personal.
- Gestionar la disponibilidad de los locales mediante horarios base, reglas por día de la semana y días bloqueados (festivos, vacaciones).

Objetivos técnicos:

- Construir la aplicación sobre un framework robusto y moderno (Symfony 7.4) siguiendo el patrón MVC.
- Garantizar la seguridad de las cuentas mediante autenticación de doble factor (2FA) por correo electrónico.
- Contenerizar toda la infraestructura con Docker para asegurar la reproducibilidad del entorno.
- Exponer una API REST con autenticación JWT para permitir integraciones o futuras aplicaciones móviles.

1.3 Alcance

El sistema cubre las siguientes áreas funcionales:

Área	Funcionalidades incluidas
Reservas	Selección de local, servicio, empleado y franja horaria disponible
Gestión de citas	Listado, confirmación, cancelación y completado de citas
Catálogo	Alta, baja lógica y edición de servicios y productos por local
Productos en cita	El cliente puede asociar productos del catálogo a su próxima cita; quedan reflejados con su precio en el perfil del cliente y en el panel de administración
Empleados	Gestión de usuarios con rol <code>ROLE_EMPLEADO</code> asignados a un local
Disponibilidad	Horarios de apertura/cierre, reglas por día de la semana, días bloqueados
Clientes	Registro, perfil, historial de citas y valoraciones
Seguridad	Login con formulario, 2FA por email, recuperación de contraseña, JWT para API
Notificaciones	Correos de recordatorio (24 h antes) y correos de confirmación/cancelación
Mensajes	Formulario de contacto con bandeja de entrada en el panel de administración

Queda **fuera del alcance** de esta versión: pasarela de pago integrada, aplicación móvil nativa y sistema de fidelización/bonos.

1.4 Público Objetivo

El sistema está orientado a dos perfiles de usuario claramente diferenciados:

Clientes finales: personas que desean reservar cita en una peluquería Venus desde cualquier dispositivo (ordenador, tablet o móvil) de forma rápida, sin necesidad de llamar por teléfono.

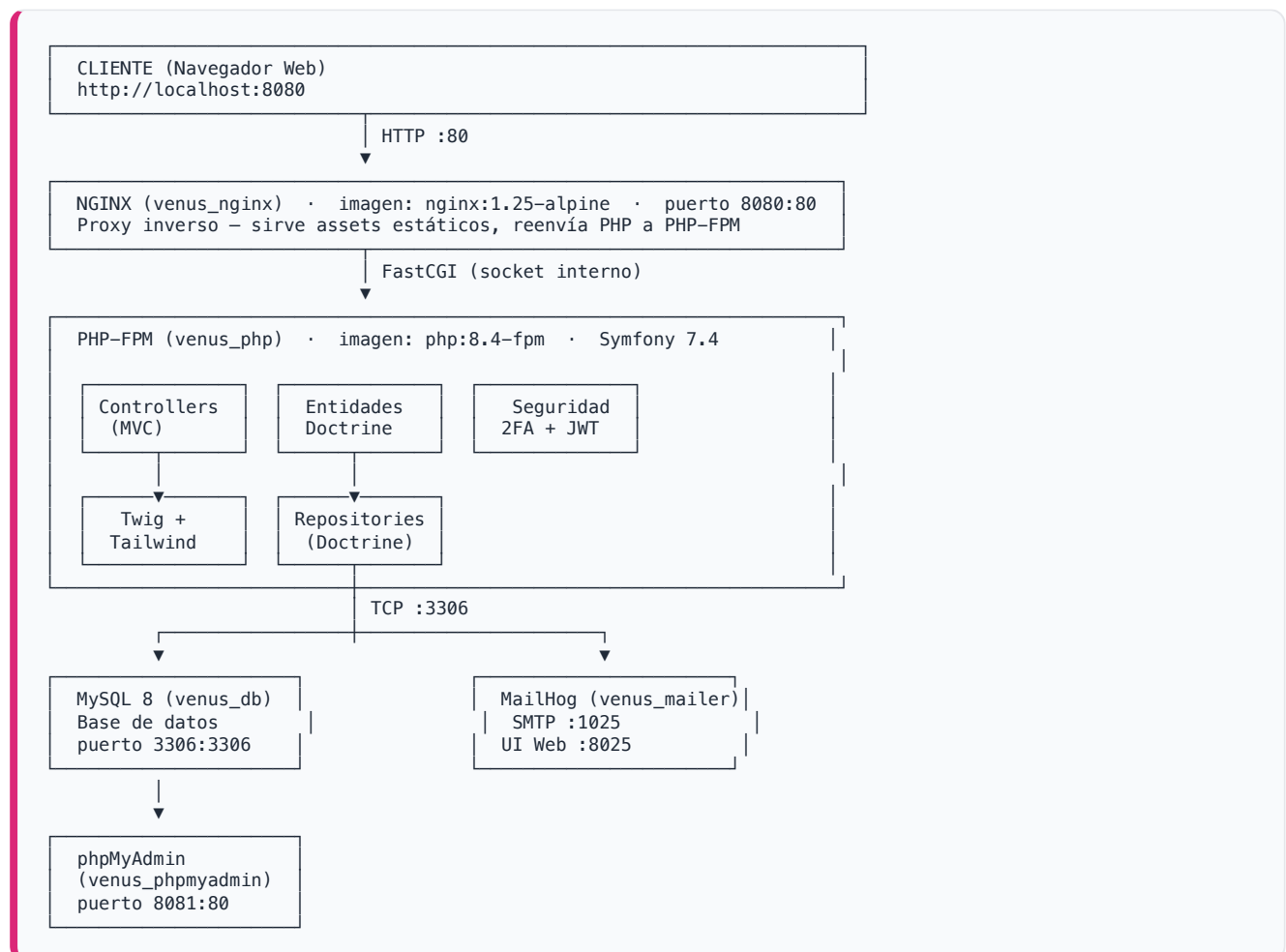
Personal del negocio:

- *Administradores* (`ROLE_ADMIN`): propietarios o encargados que necesitan una visión completa del negocio, acceso al panel de control y capacidad de configurar todos los parámetros del sistema.
- *Empleados* (`ROLE_EMPLEADO`): estilistas asignados a un local que pueden consultar sus citas programadas.

2. Arquitectura del Sistema

2.1 Diagrama de Arquitectura de Alto Nivel

La aplicación se despliega mediante Docker Compose y está compuesta por cuatro servicios que se comunican a través de una red interna (`venus_network`):



2.2 Descripción de los Componentes

Componente	Imagen Docker	Puerto externo	Función
venus_php	php:8.4-fpm (Dockerfile propio)	—	Ejecuta la aplicación Symfony vía PHP-FPM
venus_nginx	nginx:1.25-alpine	8080:80	Proxy inverso; sirve archivos estáticos y delega PHP a venus_php
venus_db	mysql:8.0	3306:3306	Persistencia de datos relacionales
venus_mailer	mailhog/mailhog	1025 (SMTP), 8025 (UI)	Captura de correos en entorno de desarrollo
venus_phpmyadmin	phpmyadmin/phpmyadmin	8081:80	Interfaz gráfica de administración de la base de datos

2.3 Patrón de Diseño: MVC en Symfony

La aplicación sigue estrictamente el patrón **Modelo-Vista-Controlador (MVC)**:



Capas de la aplicación:

- **Controladores** (`src/Controller/`): reciben peticiones HTTP, orquestan la lógica y devuelven respuestas.
- **Entidades** (`src/Entity/`): clases PHP que mapean las tablas de la base de datos mediante anotaciones de Doctrine ORM.
- **Repositorios** (`src/Repository/`): encapsulan las consultas a la base de datos mediante el patrón Repository.
- **Formularios** (`src/Form/`): gestionan la validación y el enlace de datos de formularios HTML.

- **Plantillas** (`templates/`): vistas Twig que generan el HTML final con Tailwind CSS.
 - **Seguridad** (`src/Security/`): autenticador personalizado conectado al flujo de 2FA.
 - **Comandos** (`src/Command/`): tareas CLI ejecutables desde consola (p. ej., envío de recordatorios).
-

3. Modelo de Datos

3.1 Entidades del Modelo

El modelo de datos está compuesto por las siguientes entidades:

LOCAL · USER · CITA · SERVICIO · PRODUCTO · HORARIO · REGLA_HORARIO · DIA_BLOQUEADO · MENSAJE_CONTACTO · RESET_PASSWORD_REQUEST

3.2 Relaciones entre Entidades

Entidad A	Cardinalidad	Entidad B	Descripción
LOCAL	1 : N	USER	Un local tiene muchos empleados. Un empleado pertenece a un local.
LOCAL	1 : N	CITA	Un local acoge muchas citas. Cada cita se realiza en un local.
LOCAL	1 : N	SERVICIO	Un local ofrece muchos servicios. Cada servicio pertenece a un local.
LOCAL	1 : N	HORARIO	Un local tiene muchos tramos horarios (turnos partidos).
LOCAL	1 : N	REGLA_HORARIO	Un local configura muchas reglas de horario por día de la semana.
LOCAL	1 : N	DIA_BLOQUEADO	Un local puede tener muchos días bloqueados (festivos, vacaciones).
LOCAL	N : M	PRODUCTO	Un producto puede estar disponible en varios locales. Tabla intermedia: <code>producto_local</code> .
USER	1 : N	CITA	Un cliente (usuario) puede tener muchas citas.
USER	1 : N	CITA	Un empleado puede atender muchas citas (relación opcional).
USER	1 : N	RESET_PASSWORD_REQUEST	Un usuario puede tener varias solicitudes de restablecimiento de contraseña.
CITA	N : M	SERVICIO	Una cita incluye uno o varios servicios. Tabla intermedia: <code>cita_servicio</code> .
CITA	N : M	PRODUCTO	En una cita se pueden usar uno o varios productos. Tabla intermedia: <code>cita_producto</code> .

Tablas intermedias generadas automáticamente por Doctrine:

Tabla intermedia	Entidades que relaciona
cita_servicio	CITA N:M SERVICIO
cita_producto	CITA N:M PRODUCTO
producto_local	PRODUCTO N:M LOCAL

3.3 Diccionario de Datos

Entidad User

Campo	Tipo SQL	Nulo	Descripción
id	INT AUTO_INCREMENT	NO	Clave primaria
email	VARCHAR(180)	NO	Correo electrónico; identificador de login; único
password	VARCHAR(255)	NO	Contraseña hasheada (bcrypt/argon2)
nombre	VARCHAR(255)	NO	Nombre completo del usuario
telefono	VARCHAR(20)	SÍ	Teléfono de contacto
roles	JSON	NO	Array de roles: <code>ROLE_USER</code> , <code>ROLE_ADMIN</code> , <code>ROLE_EMPLEADO</code>
auth_code	VARCHAR(255)	SÍ	Código temporal de 2FA enviado por email
local_id	INT (FK)	SÍ	Local al que pertenece el empleado (nulo en clientes)

Entidad **Cita**

Campo	Tipo SQL	Nulo	Descripción
id	INT AUTO_INCREMENT	NO	Clave primaria
fecha_inicio	DATETIME	NO	Inicio de la cita
fecha_fin	DATETIME	NO	Fin de la cita (calculado según duración de servicios)
estado	VARCHAR(20)	NO	Estado: Pendiente , Confirmada , Cancelada , Completada
notas	TEXT	SÍ	Observaciones adicionales del cliente o admin
valoracion	SMALLINT	SÍ	Puntuación del cliente (1–5) tras completar la cita
comentario_valoracion	TEXT	SÍ	Comentario libre de la valoración
usuario_id	INT (FK)	NO	Cliente que reservó la cita
empleado_id	INT (FK)	SÍ	Empleado asignado (nulo si no se eligió)
local_id	INT (FK)	NO	Local donde se realiza la cita

Entidad **Servicio**

Campo	Tipo SQL	Nulo	Descripción
id	INT AUTO_INCREMENT	NO	Clave primaria
nombre	VARCHAR(255)	NO	Nombre del servicio
duration	INT	NO	Duración en minutos
precio	DOUBLE	NO	Precio en euros
categoria	VARCHAR(100)	NO	Categoría (Peluquería, Estética, Coloración, Bienestar...)
activo	TINYINT(1)	NO	Baja lógica: 1 activo, 0 deshabilitado
local_id	INT (FK)	SÍ	Local al que pertenece el servicio

Entidad **Producto**

Campo	Tipo SQL	Nulo	Descripción
id	INT AUTO_INCREMENT	NO	Clave primaria
nombre	VARCHAR(255)	NO	Nombre del producto
marca	VARCHAR(255)	NO	Marca comercial
descripcion	TEXT	SÍ	Descripción detallada
precio	DOUBLE	NO	Precio de venta en euros
imagen	VARCHAR(255)	SÍ	Nombre de archivo de la imagen (almacenada en <code>public/uploads/productos/</code>)
stock	INT	NO	Unidades disponibles (por defecto 0)

Entidad **Local**

Campo	Tipo SQL	Nulo	Descripción
id	INT AUTO_INCREMENT	NO	Clave primaria
nombre	VARCHAR(255)	NO	Nombre comercial del local
direccion	VARCHAR(255)	NO	Dirección postal
ciudad	VARCHAR(100)	NO	Ciudad
telefono	VARCHAR(20)	SÍ	Teléfono del local
email	VARCHAR(255)	SÍ	Correo electrónico del local
activo	TINYINT(1)	NO	Estado activo/inactivo del local

Entidad **Horario**

Campo	Tipo SQL	Nulo	Descripción
id	INT AUTO_INCREMENT	NO	Clave primaria
hora_apertura	TIME	NO	Hora de apertura del turno
hora_cierre	TIME	NO	Hora de cierre del turno
intervalo_minutos	INT	NO	Granularidad de los turnos en minutos (por defecto 30)
local_id	INT (FK)	NO	Local al que pertenece este horario

Un local puede tener varios registros **Horario** para representar turnos partidos (p. ej., mañana 09:00–14:00 y tarde 16:00–20:00).

Entidad **ReglaHorario**

Campo	Tipo SQL	Nulo	Descripción
id	INT AUTO_INCREMENT	NO	Clave primaria
dia_semana	INT	NO	Día de la semana (0 = domingo, 1 = lunes, ..., 6 = sábado)
hora_desde	TIME	SÍ	Inicio del horario especial para ese día
hora_hasta	TIME	SÍ	Fin del horario especial para ese día
motivo	VARCHAR(255)	SÍ	Descripción de la regla (p. ej., "Cierre sábado tarde")
local_id	INT (FK)	NO	Local al que aplica la regla

Entidad **DiaBloqueado**

Campo	Tipo SQL	Nulo	Descripción
id	INT AUTO_INCREMENT	NO	Clave primaria
fecha	DATE	NO	Fecha de inicio del bloqueo
fecha_fin	DATE	SÍ	Fecha de fin del bloqueo (si es un rango, p. ej., vacaciones)
motivo	VARCHAR(100)	SÍ	Descripción del bloqueo (festivo, vacaciones, etc.)
local_id	INT (FK)	NO	Local al que afecta el bloqueo

Entidad MensajeContacto

Campo	Tipo SQL	Nulo	Descripción
id	INT AUTO_INCREMENT	NO	Clave primaria
nombre	VARCHAR(150)	NO	Nombre del remitente
email	VARCHAR(180)	NO	Correo del remitente
asunto	VARCHAR(255)	NO	Asunto del mensaje
mensaje	TEXT	NO	Contenido del mensaje
creado_en	DATETIME	NO	Fecha y hora de envío (se asigna automáticamente)
leido	TINYINT(1)	NO	Indica si el administrador ha leído el mensaje

Entidad ResetPasswordRequest

Campo	Tipo SQL	Nulo	Descripción
id	INT AUTO_INCREMENT	NO	Clave primaria
user_id	INT (FK)	NO	Usuario que solicita el restablecimiento
selector	VARCHAR(20)	NO	Selector público del token
hashed_token	VARCHAR(100)	NO	Hash del token de validación
requested_at	DATETIME	NO	Momento de la solicitud
expires_at	DATETIME	NO	Expiración del token (por defecto 1 hora)

4. Especificaciones Técnicas

4.1 Tecnologías Utilizadas

Capa	Tecnología	Versión
Lenguaje backend	PHP	≥ 8.2 (imagen Docker: 8.4)
Framework backend	Symfony	7.4
ORM	Doctrine ORM	^3.6
Migraciones de BD	Doctrine Migrations Bundle	^4.0
Motor de plantillas	Twig	^3.0
CSS framework	Tailwind CSS	vía <code>symfonycasts/tailwind-bundle</code> ^0.12
JavaScript reactivo	Stimulus + Turbo	<code>symfony/stimulus-bundle</code> ^2.32, <code>symfony/ux-turbo</code> ^2.32
Base de datos	MySQL	8.0
Servidor web	Nginx	1.25-alpine
Contenedores	Docker + Docker Compose	—
Autenticación	Symfony Security (formulario)	7.4
Doble factor (2FA)	<code>scheb/2fa-bundle</code> + <code>scheb/2fa-email</code>	^8.5
Dispositivos de confianza	<code>scheb/2fa-trusted-device</code>	^8.5
API REST / JWT	<code>lexik/jwt-authentication-bundle</code>	^3.2
Recuperación de contraseña	<code>symfonycasts/reset-password-bundle</code>	^1.25
Correo electrónico	Symfony Mailer	7.4
Correo en desarrollo	MailHog	—
Tests	PHPUnit	^12.5
Datos de prueba	<code>doctrine/doctrine-fixtures-bundle</code>	^4.3
Herramienta de desarrollo	Symfony Maker Bundle	^1.0

4.2 Justificación de las Tecnologías

Symfony 7.4

Symfony es uno de los frameworks PHP más maduros y utilizados a nivel profesional. Su arquitectura de componentes desacoplados, la inyección de dependencias nativa, el sistema de seguridad robusto y el amplio ecosistema de bundles lo convierten en la opción más adecuada para un proyecto de gestión empresarial. La versión 7.4 es la rama de soporte a largo plazo (LTS) activa en el momento del desarrollo.

Doctrine ORM

Permite trabajar con la base de datos mediante objetos PHP (entidades) en lugar de SQL directo, lo que reduce errores, facilita las migraciones evolutivas y desacopla la aplicación del motor de base de datos concreto.

Twig + Tailwind CSS

Twig es el motor de plantillas oficial de Symfony, con herencia de layouts, escapado automático y extensibilidad. Tailwind CSS proporciona utilidades atómicas de CSS que permiten construir interfaces responsivas y consistentes sin escribir hojas de estilo personalizadas, reduciendo el tamaño del CSS en producción mediante purge automático.

Stimulus + Turbo (Hotwire)

Estos dos microframeworks de JavaScript permiten añadir interactividad y navegación tipo SPA (reemplazos parciales de la página) sin necesidad de un framework JavaScript complejo como React o Vue, manteniendo el control del HTML en el servidor (Twig).

scheb/2fa-bundle

La autenticación de doble factor añade una capa de seguridad crítica para una aplicación que gestiona datos personales de clientes. El bundle se integra nativamente con el sistema de seguridad de Symfony y envía el código OTP por email a través de Symfony Mailer.

lexik/jwt-authentication-bundle

Permite exponer una API REST stateless con autenticación mediante tokens JWT, pensada para futuras integraciones (aplicación móvil, integraciones con terceros).

Docker Compose

Garantiza que el entorno de desarrollo sea idéntico en cualquier máquina (reproducibilidad). Simplifica el arranque de la pila completa (PHP, Nginx, MySQL, MailHog) con un único comando.

MailHog

Servidor SMTP ficticio para desarrollo que captura todos los correos salientes y los muestra en una interfaz web en <http://localhost:8025>, sin riesgo de enviar correos reales a clientes durante el desarrollo.

4.3 Dependencias Clave

Las dependencias se gestionan a través de Composer (`composer.json`). A continuación se detallan las más relevantes:

Dependencias de producción (`require`):

Paquete	Propósito
<code>symfony/framework-bundle</code>	Núcleo del framework Symfony
<code>symfony/security-bundle</code>	Sistema de autenticación y autorización
<code>symfony/form</code> + <code>symfony/validator</code>	Formularios y validación de datos
<code>symfony/mailer</code>	Envío de correos electrónicos
<code>symfony/twig-bundle</code>	Motor de plantillas
<code>symfony/ux-turbo</code> + <code>symfony/stimulus-bundle</code>	Interactividad frontend sin SPA
<code>symfony/asset-mapper</code>	Gestión de assets JS/CSS sin bundler
<code>symfonycasts/tailwind-bundle</code>	Integración de Tailwind CSS con Symfony
<code>doctrine/doctrine-bundle</code> + <code>doctrine/orm</code>	ORM y acceso a base de datos
<code>doctrine/doctrine-migrations-bundle</code>	Migraciones evolutivas de esquema
<code>scheb/2fa-bundle</code> + <code>scheb/2fa-email</code> + <code>scheb/2fa-trusted-device</code>	Autenticación 2FA por email
<code>lexik/jwt-authentication-bundle</code>	Autenticación JWT para la API REST
<code>symfonycasts/reset-password-bundle</code>	Flujo de recuperación de contraseña por token

Dependencias de desarrollo (`require-dev`):

Paquete	Propósito
<code>doctrine/doctrine-fixtures-bundle</code>	Carga de datos de prueba (fixtures)
<code>phpunit/phpunit</code>	Framework de tests unitarios e integración
<code>symfony/maker-bundle</code>	Generación de código mediante comandos <code>make:*</code>
<code>symfony/web-profiler-bundle</code>	Barra de depuración y perfilador de rendimiento

5. Manual de Despliegue

Este manual describe el proceso completo para levantar el proyecto Venus en un entorno local de desarrollo desde cero.

5.1 Prerrequisitos

Antes de comenzar, asegúrate de tener instalados los siguientes programas en tu equipo:

Herramienta	Versión mínima	Verificación
Docker Desktop	24.x	<code>docker --version</code>
Docker Compose	v2.x (integrado en Docker Desktop)	<code>docker compose version</code>
Git	2.x	<code>git --version</code>

No es necesario tener PHP, Composer ni Node.js instalados localmente; todo se ejecuta dentro de los contenedores Docker.

5.2 Configuración de Variables de Entorno

El proyecto incluye el archivo `.env` con valores por defecto para el entorno Docker de desarrollo. No es necesario modificarlo para el primer arranque.

Para personalizaciones locales (sin afectar al repositorio), puedes crear un archivo `.env.local`:

```
# Ejemplo de .env.local (opcional)
APP_SECRET=mi_secreto_personalizado
```

Los valores relevantes del `.env` para el entorno Docker son:

```
APP_ENV=dev
DATABASE_URL=mysql://app:app@database:3306/peluqueria_venus?serverVersion=8.0&charset=utf8mb4
MAILER_DSN=smtp://mailer:1025
```

La variable `DATABASE_URL` apunta al servicio `database` de Docker Compose, con usuario `app`, contraseña `app` y base de datos `peluqueria_venus`. Estos valores coinciden con las variables de entorno del contenedor MySQL en `compose.yaml`.

5.3 Construir e Iniciar los Contenedores

Desde la raíz del proyecto, ejecuta:

```
docker compose up -d --build
```

- El flag `--build` fuerza la construcción de la imagen de PHP la primera vez (o cuando cambia el `Dockerfile`).
- El flag `-d` arranca los contenedores en segundo plano (modo detached).

Verificar que los contenedores están activos:

```
docker compose ps
```

Debes ver los cinco servicios con estado `Up` o `running` :

NAME	IMAGE	STATUS
venus_php	peluqueria-php	Up
venus_nginx	nginx:1.25-alpine	Up
venus_db	mysql:8.0	Up (healthy)
venus_mailer	mailhog/mailhog	Up
venus_phpmyadmin	phpmyadmin/phpmyadmin	Up

El contenedor `venus_db` tiene configurado un healthcheck. El contenedor `venus_php` espera a que MySQL esté disponible antes de arrancar (`depends_on: database: condition: service_healthy`). Si MySQL tarda en arrancar la primera vez, espera unos segundos y vuelve a verificar.

5.4 Instalar Dependencias PHP

Si es la primera vez que arrancas el proyecto (o si el `Dockerfile` no ha instalado las dependencias en el build), ejecuta Composer dentro del contenedor PHP:

```
docker compose exec php composer install
```

En el flujo normal, las dependencias ya se instalan durante la construcción de la imagen Docker (`RUN composer install` en el `Dockerfile`), por lo que este paso puede no ser necesario.

5.5 Ejecutar las Migraciones de Base de Datos

Las migraciones crean y actualizan el esquema de la base de datos. Ejecuta:

```
docker compose exec php bin/console doctrine:migrations:migrate
```

Symfony preguntará una confirmación antes de ejecutar. Escribe **yes** y pulsa Enter.

```
WARNING! You are about to execute a migration in database "peluqueria_venus" [...]
Are you sure you wish to execute this migration? (yes/no) [yes]: yes
```

Tras ejecutar las migraciones, la base de datos contendrá todas las tablas necesarias pero estará vacía de datos.

Verificar el estado de las migraciones:

```
docker compose exec php bin/console doctrine:migrations:status
```

Todos los registros deben aparecer como **migrated**.

5.6 Cargar Datos de Prueba (Fixtures)

Las fixtures poblán la base de datos con datos de ejemplo para poder probar la aplicación:

```
docker compose exec php bin/console doctrine:fixtures:load
```

Atención: este comando **borra todos los datos existentes** en la base de datos antes de cargar los fixtures. Úsalo exclusivamente en entornos de desarrollo.

Escribe **yes** para confirmar cuando se solicite.

Datos cargados por las fixtures:

Tipo	Datos de ejemplo
Locales	Venus Alcalá (C/ Ecuador 21) y Venus Arabial (C/ Arabial 110, Granada)
Servicios	5 servicios para Local 1 y 3 para Local 2
Productos	3 productos (Champú, Mascarilla, Cera)
Horarios	Turno partido 09:00–14:00 / 16:00–20:00 (Local 1) y continuo 10:00–19:00 (Local 2)
Empleados	merce@venus.com (ADMIN+EMPLEADO), laura@venus.com , carlos@venus.com , ana@venus.com
Clientes	antonio@gmail.com , sara@gmail.com , pedro@gmail.com
Citas	3 citas futuras (una con un producto asociado) y 3 citas pasadas con valoración y comentario

Credenciales de acceso tras cargar fixtures:

Usuario	Email	Contraseña	Rol
Merce (admin)	merce@venus.com	venus123	ROLE_ADMIN + ROLE_EMPLEADO
Laura	laura@venus.com	venus123	ROLE_EMPLEADO
Carlos	carlos@venus.com	venus123	ROLE_EMPLEADO
Ana	ana@venus.com	venus123	ROLE_EMPLEADO
Antonio	antonio@gmail.com	cliente123	ROLE_USER
Sara	sara@gmail.com	cliente123	ROLE_USER
Pedro	pedro@gmail.com	cliente123	ROLE_USER

El inicio de sesión requiere completar el **segundo factor de autenticación (2FA)**. Tras introducir la contraseña, se enviará un código al correo del usuario. En desarrollo, este correo se puede consultar en la interfaz de MailHog: <http://localhost:8025>

5.7 Compilar los Assets de Tailwind CSS

Para que los estilos de Tailwind CSS se compilen correctamente, ejecuta:

```
# Una sola compilación (para pruebas puntuales)
docker compose exec php bin/console tailwind:build

# Modo watch (recompila automáticamente al modificar plantillas)
docker compose exec php bin/console tailwind:build --watch
```

En el entorno de desarrollo (`APP_ENV=dev`), Symfony Asset Mapper sirve el CSS de Tailwind directamente. Este paso es necesario si los estilos no se aplican correctamente o tras añadir nuevas clases de Tailwind en las plantillas.

5.8 Verificación del Entorno

Tras completar todos los pasos anteriores, el entorno estará completamente operativo. Verifica accediendo a las siguientes URLs:

Servicio	URL	Descripción
Aplicación Venus	http://localhost:8080	Aplicación principal
Panel de administración	http://localhost:8080/admin	Requiere login con <code>merce@venus.com</code>
MailHog (correos)	http://localhost:8025	Buzón de correos de desarrollo
phpMyAdmin	http://localhost:8081	Gestión visual de la base de datos

Flujo de prueba recomendado:

1. Accede a <http://localhost:8080> y haz clic en "Reservar cita" para probar el flujo de reserva como cliente anónimo.
2. Regístrate con un nuevo correo o inicia sesión con `antonio@gmail.com` / `cliente123`.
3. Comprueba en MailHog (<http://localhost:8025>) el correo con el código 2FA.
4. Cierra sesión e inicia sesión como administrador con `merce@venus.com` / `venus123`.
5. Explora el panel de administración en <http://localhost:8080/admin>.

5.9 Comandos de Administración Habituales

```
# Limpiar la caché de Symfony
docker compose exec php bin/console cache:clear

# Generar una nueva migración tras modificar entidades
docker compose exec php bin/console doctrine:migrations:diff

# Ejecutar los tests
docker compose exec php bin/phpunit

# Ejecutar un test específico
docker compose exec php bin/phpunit tests/Controller/CitaControllerTest.php

# Enviar recordatorios de citas del día siguiente (configurar como cron en producción)
docker compose exec php bin/console app:enviar-recordatorios

# Abrir una shell interactiva dentro del contenedor PHP
docker compose exec php bash
```

5.11 Apagar el Entorno

Para detener los contenedores sin eliminar los datos persistentes (volúmenes de MySQL):

```
docker compose down
```

Para detener y **eliminar todos los datos** (volúmenes incluidos):

```
docker compose down -v
```

Usa **down -v** con precaución, ya que eliminará permanentemente la base de datos y tendrás que volver a ejecutar las migraciones y los fixtures.